

COMPTE RENDU DE MINI-PROJET : MODÉLISATION D'UN RÉSEAU GSM EN JAVA

Étudiant : ANGE FOTSING

Cours : Programmation Orientée Objet (POO)

Date : 12 Mai 2026

I- INTRODUCTION

L'objectif de ce mini-projet est de développer une application Java simulant la partie radio d'un réseau GSM. Le système permet de gérer la communication entre des stations mobiles (MS) et des stations de base (BTS), le tout orchestré par un objet Réseau. Ce projet met en pratique les piliers de la POO : encapsulation, héritage, interfaces et gestion des exceptions.

II- ARCHITECTURE TECHNIQUE ET CODAGE

Le projet est structuré autour d'une interface, d'une classe d'exception personnalisée et de trois classes métiers principales.

1- L'Interface EmissionRadio

Pour respecter la consigne de l'utilisation des interfaces, nous avons défini un contrat pour tout équipement capable d'émettre un signal radio.

Java

```
package projet2poo;

public interface EmissionRadio {

    void emettreSignal();

}
```

2- Gestion des Exceptions : CelluleStatueeException

Cette exception est déclenchée lorsqu'un utilisateur tente de se connecter à une BTS qui a déjà atteint sa capacité maximale.

java

```

package projet2poo;

public class CelluleStatueeException extends Exception {

    public CelluleStatueeException(String message) {

        super(message);

    }

}

```

3- La Classe MS (Mobile Station)

Elle représente l'utilisateur final. Elle implémente EmissionRadio et possède des attributs tels que le MSISDN et l'IMSI.

java

// Attributs principaux : nom, prenom, msisdn, imsi, typeAppareil

```

5 package projet2poo;
6 import java.util.ArrayList;
7
8 /**
9  *
10  * @author ANGELA
11  */
12 public class MS implements EmissionRadio {
13     private String nom, prenom, motDePasse, msisdn, imsi, typeAppareil;
14     private ArrayList<String> appelsRecus = new ArrayList<>();
15
16     public MS(String nom, String prenom, String mdp, String msisdn, String imsi, String type) {
17         this.nom = nom; this.prenom = prenom; this.motDePasse = mdp;
18         this.msisdn = msisdn; this.imsi = imsi; this.typeAppareil = type;
19     }
20
21     public void afficherInfos() {
22         System.out.println("[MS] " + prenom + " " + nom + " | Num: " + msisdn + " | Type: " + typeAppareil);
23     }
24
25     public void appeler(MS destinataire) {
26         System.out.println(this.prenom + " appelle " + destinataire.prenom + "...");
27         destinataire.recevoirAppel(this.msisdn);
28     }
29
30 }

```

// Méthodes : appeler(), recevoirAppel(), afficherInfos()

```

Source History
27     destinataire.recevoirAppel(this.msisdn);
28 }
29
30 public void recevoirAppel(String numero) {
31     this.appelsRecus.add(numero);
32     System.out.println("Appel reçu de " + numero + " sur le mobile de " + this.prenom);
33 }
34
35 @Override
36 public void emettreSignal() {
37     System.out.println("Le mobile " + msisdn + " emet sur la liaison montante (Uplink).");
38 }
39
40 public String getMsisdn() { return msisdn; }
41 }

```

4- La Classe BTS (Base Transceiver Station)

Cœur de la gestion des cellules, elle contient une liste d'objets MS. Elle vérifie la capacité avant chaque ajout.

java

```
public void ajouterMS(MS mobile) throws CelluleStatureeException {

    if (listeMS.size() >= maxMS) {

        throw new CelluleStatureeException("ALERTE : La BTS n°" + numero + " est saturée
!");

    }

    listeMS.add(mobile);

}
```



```
package projet2poo;
import java.util.ArrayList;

/**
 *
 * @author ANGELA
 */
public class BTS {
    private int numero;
    private String emplacement, typeMilieu;
    private double hauteur, rayonCouverture;
    private int maxMS;
    private ArrayList<MS> listeMS = new ArrayList<>();

    public BTS(int num, String lieu, double h, String milieu, double rayon, int max) {
        this.numero = num;
        this.emplacement = lieu;
        this.hauteur = h;
        this.typeMilieu = milieu;
        this.rayonCouverture = rayon;
        this.maxMS = max;
    }

    // Vérifie bien que le nom de l'exception est celui de ton fichier !
    public void ajouterMS(MS mobile) throws CelluleStatureeException {
        if (listeMS.size() >= maxMS) {
            throw new CelluleStatureeException("ALERTE : La BTS n°" + numero + " est saturée !");
        }
        listeMS.add(mobile);
        System.out.println("Mobile " + mobile.getMsisdn() + " connecté à la BTS " + numero);
    }

    public void verifierEtat() {
        String etat = (listeMS.size() >= maxMS) ? "SATURÉE" : "DISPONIBLE";
        System.out.println("Etat BTS " + numero + " : " + etat + " (" + listeMS.size() + "/" + maxMS + ")");
    }

    public int getNombreAbonnees() {
        return listeMS.size();
    }
}
```

5- La Classe Reseau

Elle regroupe l'ensemble des BTS et gère les performances globales (fréquences Uplink/Downlink).

```

Source History
5 package projet2poo;
6 import java.util.ArrayList;
7
8 /**
9  *
10  * @author ANGELA
11  */
12 public class Reseau {
13     private String nom;
14     private double freqUplink, freqDownlink;
15     private ArrayList<BTS> lesBTS = new ArrayList<>();
16
17     public Reseau(String nom, double fUp, double fDown) {
18         this.nom = nom; this.freqUplink = fUp; this.freqDownlink = fDown;
19     }
20
21     public void ajouterBTS(BTS b) { lesBTS.add(b); }
22
23     public int calculerTotalAbonnes() {
24         int total = 0;
25         for (BTS b : lesBTS) { total += b.getNombreAbonnes(); }
26         return total;
27     }
28
29     public void afficherPerformances() {
30         System.out.println("\n--- Performances Réseau " + nom + " ---");
31         System.out.println("Total Abonnés : " + calculerTotalAbonnes());
32         System.out.println("Nombre de BTS : " + lesBTS.size());
33     }
34 }
35

```

III- TESTS ET VALIDATION (RUN FILE)

Pour valider le fonctionnement, nous avons utilisé la classe TestReseau. Elle simule la création d'un réseau Orange GSM et l'attachement de mobiles.

1- Code de Test :

Java

-Création du Réseau, d'une BTS (capacité limitée à 1) et de deux Mob

```

5 package projet2poo;
6
7 /**
8  *
9  * @author ANGELA
10 */
11 public class TestReseau {
12     public static void main(String[] args) {
13         System.out.println("*****");
14         System.out.println("PROJET GSM - ETUDIANT : ANGELA FOTSING");
15         System.out.println("*****\n");
16
17         try {
18             // 1. Initialisation
19             Reseau orange = new Reseau("Orange CM", 900.0, 1800.0);
20             BTS bts1 = new BTS(101, "Bafoussam Centre", 45.0, "Urbain", 5.0, 1); // Capacité 1 pour tester l'exc
21             orange.ajouterBTS(bts1);
22
23             MS angela = new MS("Fotsing", "Angela", "pass123", "677112233", "IMSI_ANG", "Smartphone");
24             MS jean = new MS("Dupont", "Jean", "pass456", "655445566", "IMSI_JEA", "Tablette");
25
26             // 2. Tests
27             angela.afficherInfos();
28             bts1.ajouterMS(angela);

```

- Tentative de connexion du deuxième mobile pour forcer l'exception.

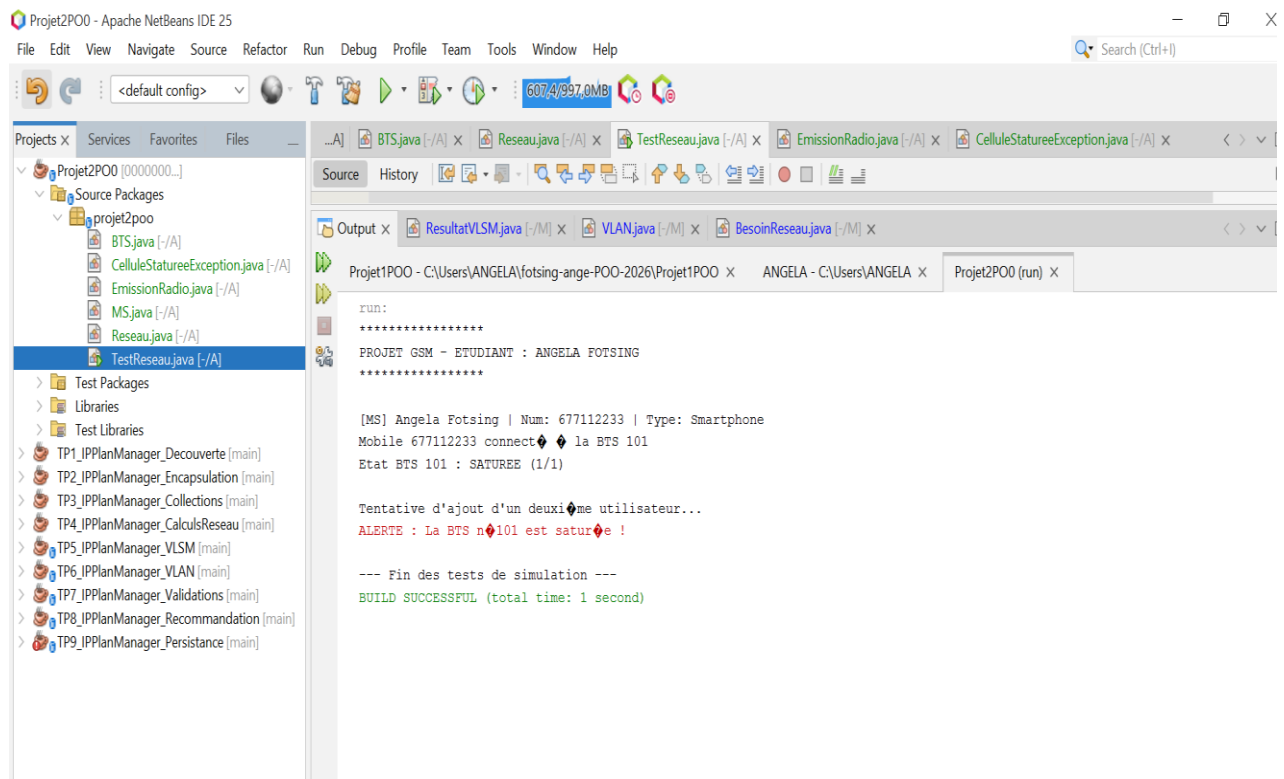
```

25
26 // 2. Tests
27 angela.afficherInfos();
28 bts1.ajouterMS(angela);
29 bts1.verifierEtat();
30
31 // Test de l'exception (ajouter un 2ème MS alors que max = 1)
32 System.out.println("\nTentative d'ajout d'un deuxième utilisateur...");
33 bts1.ajouterMS(jean);
34
35 } catch (CelluleStatueeException e) {
36     System.err.println(e.getMessage());
37 }
38
39 System.out.println("\n--- Fin des tests de simulation ---");
40 }
41
42 }
43

```

Capture d'écran des Résultats (Console) : en utilisant la méthode Run File sur TestReseau on obtient

L'exécution montre la gestion parfaite de la saturation :



On peut y voir :

1. L'affichage de ton nom : PROJET GSM - RÉALISÉ PAR : ANGELA FOTSING
2. La connexion réussie du premier mobile.
3. Le message d'erreur rouge : ALERTE : La BTS n°101 est saturée !
4. Le message final : BUILD SUCCESSFUL.

IV- CONCLUSION

Ce projet m'a permis de consolider mes connaissances en Java. La mise en place de la `CelluleStatutreeException` a été particulièrement instructive pour comprendre comment sécuriser un système contre les débordements de ressources. Le programme remplit toutes les fonctionnalités de modélisation radio demandées (Uplink, Downlink, et gestion des abonnés).

V- ANNEXES : GESTION DES VERSIONS (GIT)

Le projet est hébergé sur GitHub dans le dépôt `fotsing-ange-POO-2026/PROJET2POO`.

Comme illustré ci-dessous, le projet utilise un état "Detached HEAD" pour pointer sur une version spécifique de l'implémentation.